

P3983R0

Object Representation for `std::simd`

Published Proposal, 2026-01-27

Author:

[Daniel Towner](#) (Intel)

Audience:

LWG, LEWG

Project:

ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Abstract

The Working Draft specification of `std::simd` makes `basic_vec` trivially copyable, enabling `std::bit_cast`, but leaves the object representation unspecified. This prevents portable bit reinterpretation idioms that are widely used in SIMD code and supported by existing intrinsic APIs. This paper proposes two approaches: specifying an array-like object representation for `basic_vec<T, native_abi<T>>` with no inter-element or trailing padding, or alternatively adding traits for implementations to report whether a given specialization is array-like. The paper recommends specifying array-like layout for `native_abi` specializations to provide portable semantics with minimal user burden. The `fixed_size` ABI is explicitly left for future work.

Table of Contents

- 1 Introduction
- 2 Motivation
- 3 Proposed Solution 1: Mandate Array-Like Layout for `native_abi`
 - 3.1 Why This Matches Hardware Reality
 - 3.2 The Standard Already Strongly Implies Array-Like Layout

3.3 Why `fixed_size` Is Excluded

3.4 Handling Missing Hardware Support

4 Proposed Solution 2: Query Traits

5 Complementary Use of Both Solutions

6 Comparison

7 Our Recommendation

8 Additional Discussion

8.1 Implementation Experience

8.2 Scalar Type Consistency

8.3 Endianness

8.4 Library Interoperability

8.5 ABI Stability

8.6 Complex Numbers

8.7 Masks

9 Proposed Wording

9.1 Wording for Recommended Approach (Combined)

9.2 Wording for Solution 2 Alone (Fallback)

10 Impact on Existing Code

11 Future Work

§ 1. Introduction

The Working Draft makes `simd` types `TriviallyCopyable`, which allows `std::bit_cast` operations. However, the object representation is unspecified, making the results implementation-defined.

```
vec<float> v = {...};  
auto as_int = std::bit_cast<vec<int32_t>>(v);  
// Legal C++26, but what are the integer values?
```

This contrasts with `std::array`, which has a well-specified contiguous layout that makes `bit_cast` operations portable and predictable. Since `basic_vec<T, N>` is conceptually similar to `array<T, N>` (both are fixed-size containers of homogeneous elements), users naturally expect similar bit-casting guarantees. Without specifying the layout, such code is not portable across implementations.

Furthermore, all target-specific intrinsics (e.g., Intel’s `_mm256_castps_si256`, ARM’s `vreinterpretq_s32_f32`, etc.) provide well-defined bit-reinterpretation. Users migrating from intrinsics to `std::simd` lose this capability because the Working Draft does not specify an object representation that gives portable semantics for such reinterpretation.

This paper proposes mandating array-like object representation specifically for `basic_vec<T, native_abi<T>>`, where the guarantee is clean, unambiguous, and aligned with universal hardware practice. For other ABIs — including `fixed_size` and implementation-defined ABIs — the object representation remains implementation-defined. The paper also offers a fallback solution based on query traits, and notes that the two approaches are complementary.

§ 2. Motivation

Bit-level operations on SIMD vectors are pervasive in performance-critical code. Common patterns include clearing sign bits for fast absolute value, inspecting IEEE 754 exponent bits, and type-punning between float and integer vectors. For example:

```
template<typename T, typename Abi>
auto abs_via_bitwise(basic_vec<T, Abi> v) -> basic_vec<T, Abi> {
    auto bits = std::bit_cast<basic_vec<uint_t, Abi>>(v);
    bits &= ~sign_bit_mask;
    return std::bit_cast<basic_vec<T, Abi>>(bits);
}
```

Similar bit-casting patterns appear across a wide range of domains, including scientific computing, signal processing, game engines, and numeric libraries. Every target vendor provides these operations with well-defined semantics (e.g., Intel’s `_mm256_castps_si256`, ARM’s `vreinterpretq_s32_f32`). Furthermore, the bit pattern meanings are consistent across mainstream intrinsic APIs, so vendor intrinsic code already works portably. However, because the object representation of `basic_vec` is not specified, these idioms do not have portable semantics when expressed in terms of `std::simd`.

The case for specifying layout is strongest for `native-abi<T>`. This is the default ABI which maps directly to hardware registers, and it is the ABI that corresponds to vendor intrinsic types. When developers use `native-abi`, they are explicitly requesting the hardware's natural representation, and they expect the bit-level semantics that come with it.

In production code for high-performance signal processing, predictable bit-level behavior is essential. When a developer `bit_cast`s a SIMD vector, the expectation is the same semantics provided by vendor intrinsics or by working with arrays. This expectation is well-founded: it reflects decades of consistent hardware design across all major CPU architectures.

If an implementation were to choose a non-standard layout, several problems would arise:

- The ability to reason about bit-level operations would be lost
- Optimisation techniques that are standard practice with intrinsics would become unusable
- Defensive code with fallback paths would be required to accommodate one implementation's unusual choice

The problem is not with flexibility per se, but rather that the exceptional non-standard case penalises the common case of writing portable code. If an implementation genuinely needs a different layout, that is what custom ABI tags are for. Such exceptional choices should be explicit and discoverable, not hidden behind implementation-defined behavior that forces every user to write defensive code.

We now examine two possible solutions in more detail.

§ 3. Proposed Solution 1: Mandate Array-Like Layout for `native-abi`

This solution requires that `basic_vec<T, native-abi<T>>` stores `size()` values of type `T` contiguously and in index order, with no inter-element padding and no trailing padding. The object representation is identical to `array<T, size()>`. This layout matches the behavior of all mainstream SIMD targets we are aware of, and is well-suited to SIMD processing.

```
template<typename T>
auto abs_via_bitwise(basic_vec<T, native-abi<T>> v)
    -> basic_vec<T, native-abi<T>> {
    auto bits = std::bit_cast<basic_vec<uint_t, native-abi<uint_t>>>(v);
    bits &= ~sign_bit_mask;
    return std::bit_cast<basic_vec<T, native-abi<T>>>(bits);
}
// Works everywhere with native-abi.
```

This approach provides guaranteed portability for native-width vectors, straightforward user code, and matches the behavior of all mainstream SIMD targets we are aware of.

This paper is specifically about object representation in support of portable `std::bit_cast` (and `memcpy`-style copying). It does not propose pointer-interconvertibility with `T*`, nor does it propose any change to existing aliasing or lifetime rules.

§ 3.1. Why This Matches Hardware Reality

Across all mainstream SIMD targets we are aware of (including Intel/AMD x86, Arm NEON/SVE, RISC-V V, and PowerPC/VSX), vector data is naturally treated as a contiguous sequence of elements when transferred to and from memory, and the mapping between element index and increasing memory offset is consistent with array-like layout. This is the layout that existing vendor intrinsics and idioms assume.

For `native_abi<T>` specifically, the vector maps to a single hardware register, and the elements fill that register completely with no trailing padding. This makes the array-like guarantee clean and unambiguous.

§ 3.2. The Standard Already Strongly Implies Array-Like Layout

The existing Working Draft contains multiple indications that array-like layout is expected, and these indications are particularly strong for `native_abi`:

The meaning of "native": The specification says `native_abi<T>` should provide "the most efficient data-parallel execution for the element type T on the currently targeted system" and that representation should "depend on the target architecture." The word "native" means the hardware's natural representation. Scoping the layout mandate to `native_abi` is therefore directly aligned with the specification's own intent: if the ABI is the native one, the layout should be the native one too.

Recommended practice for conversions: The draft states "Implementations should support implicit conversions between specializations of `basic_vec` and appropriate implementation-defined types (see *simd.overview*(<https://eel.is/c++draft/simd#overview>))." These implementation-defined types are vendor intrinsics like `__m256`. This creates an inconsistency:

```
// This is legal and well-defined at every step
basic_vec<float, native_abi<float>> v = /* ... */;
__m256 native = v;                                // Recommended conversion
__m256i as_int = _mm256_castps_si256(native);      // Well-defined intrinsic
```

```
basic_vec<int, native_abi<int>> result = as_int; // Recommended conversion

// But this equivalent direct path is implementation-defined
auto result = std::bit_cast<basic_vec<int, native_abi<int>>>(v);
```

The indirect path through intrinsics is legal and portable because intrinsics have well-defined bit-reinterpretation semantics. But the direct path is not portable. If intrinsic interop is recommended, then the layout implications of that interop should also be normative; otherwise the recommendation is misleading.

ABI tags handle variation: The ABI parameter mechanism exists precisely to handle platform-specific variations. Scoping the mandate to `native_abi` respects this design: the native ABI gets a firm guarantee, while custom and implementation-defined ABIs retain full freedom.

§ 3.3. Why `fixed_size` Is Excluded

The `fixed_size<T, N>` ABI presents additional challenges that make it unsuitable for the same guarantee. When the requested number of elements does not match a single hardware register width, implementations must decompose the logical vector into multiple hardware registers. Different implementations make legitimately different choices about how to perform this decomposition:

- Some implementations round up to the next power of two (e.g., GCC)
- Others pack into as many full-width registers as possible, with a smaller trailing register (e.g., Clang)

These choices affect `sizeof`, internal alignment boundaries, and trailing padding. Mandating a specific layout for `fixed_size` would either constrain implementations unnecessarily or require a trailing-padding escape hatch that undermines the portability guarantee.

However, we observe that on a given implementation, `fixed_size` specializations whose element data requires the same total number of bits will typically use the same register decomposition. For example, `basic_vec<uint32_t, fixed_size<22>>` and `basic_vec<uint64_t, fixed_size<11>>` both require 704 bits of element data. An implementation targeting a platform with 512-bit registers might represent both as one 512-bit register plus one 256-bit register (with 64 bits of padding). Because the decomposition is the same, the element data occupies the same byte offsets within the object representation, and `bit_cast` between the two types would produce meaningful results.

Notably, users can already test whether two types are `bit_cast`-compatible at compile time, since `std::bit_cast` requires equal `sizeof`:

```
static_assert(sizeof(basic_vec<uint32_t, fixed_size<22>>) ==
              sizeof(basic_vec<uint64_t, fixed_size<11>>));
// If this passes, bit_cast compiles.
```

This size equality is a necessary condition for `bit_cast`, but it is not sufficient for portable element-wise reinterpretation — that additionally requires knowing that elements are laid out contiguously and in index order. For `native_abi`, this paper mandates that property. For `fixed_size`, the layout is implementation-defined, so while `bit_cast` between same-sized `fixed_size` specializations is likely to work on any given platform, it is not portably guaranteed.

This also highlights an important distinction: for `fixed_size` vectors, the most useful guarantee may not be "is this type array-like?" (a per-type property) but rather "are these two types layout-compatible with each other?" (a pairwise property). Two `fixed_size` vectors might both have internal padding that prevents either from being array-like, yet share identical internal structure, making `bit_cast` between them well-defined.

A robust specification for `fixed_size` layout, including the question of pairwise compatibility, is a more complex problem that deserves its own treatment. Although the layout guarantee proposed here applies to `native_abi<T>`, users working with `fixed_size` vectors can use the `chunk` function to decompose into native-sized pieces, each of which carries the array-like guarantee. A complete solution for `fixed_size` bit-casting, including handling of remainder chunks, is left for future work.

§ 3.4. Handling Missing Hardware Support

Implementations already handle missing hardware support without altering layouts. Even modern Intel processors do not provide a completely uniform instruction set for all possible data types. For example, AVX-512 lacks 8-bit integer multiplication, shift, and rotate instructions. Rather than adopting a special layout for `basic_vec<int8_t, native_abi<int8_t>>`, implementations synthesize the specific operations that are missing using wider operations (e.g., 16-bit multiplication with masking) or scalar fallbacks, while maintaining the same array-like layout.

When hardware lacks native support for a type, `native_abi<T>` may fall back to software emulation that still uses array-like layout. The guarantee holds for whatever `native_abi<T>` is defined to be. When hardware does not support operations natively, implementations maintain the standard layout and emulate operations. They do not change the memory representation.

§ 4. Proposed Solution 2: Query Traits

As a fallback to mandating array-like layout, implementations could instead be required to document their layout and provide a compile-time query:

```
template<typename T, typename Abi>
inline constexpr bool is_simd_array_like_v = /* implementation-defined */;
```

This trait indicates whether `basic_vec<T, Abi>` has contiguous, index-ordered layout with no inter-element or trailing padding. Users could then write defensive code with `static_assert`, or adaptive code with fast and slow paths for different layouts. The trait takes both `T` and `Abi` because layout may depend on element type as well as the ABI.

Just as the standard provides `std::endian` to query byte order, this solution would provide a way to query the layout of `basic_vec` or `basic_mask` types, enabling users to determine at compile time whether the layout is array-like.

This solution preserves maximum implementation freedom but does not guarantee portability. Users must write more complex code, and generic libraries need conditional compilation with potential performance cliffs.

§ 5. Complementary Use of Both Solutions

Solutions 1 and 2 are not mutually exclusive. One outcome may be to adopt both:

- **Solution 1** mandates array-like layout for `native_abi`, giving users a firm, unconditional guarantee for the most common case.
- **Solution 2** provides `is_simd_array_like_v` as a query trait for all other ABIs (including `fixed_size` and implementation-defined ABIs), enabling generic code to adapt to whatever layout the implementation provides.

Under this combined approach, `is_simd_array_like_v<T, native_abi<T>>` would be unconditionally `true` (as a consequence of the mandate), while for other ABIs it would be implementation-defined but discoverable. This gives users the best of both worlds: zero-overhead portable code for native-width vectors, and a principled way to handle other ABIs without sacrificing generality.

§ 6. Comparison

Aspect	Solution 1 (Mandate for <code>native-abi</code>)	Solution 2 (Query)	Combined
Scope	<code>native-abi</code> only	All ABIs via query	<code>native-abi</code> guaranteed; others queryable
Portability	Guaranteed for <code>native-abi</code>	Conditional	Guaranteed for <code>native-abi</code> ; conditional for others
Trailing Padding	None (for <code>native-abi</code>)	Implementation-defined	None for <code>native-abi</code> ; implementation-defined for others
User Code	Simple for <code>native-abi</code>	Complex (conditionals)	Simple for <code>native-abi</code> ; conditionals for others
Implementation Freedom	Constrained for <code>native-abi</code> ; full freedom for other ABIs	Maximum	Constrained for <code>native-abi</code> ; documented for others
Zero-Overhead for Portable Code	Yes (for <code>native-abi</code>)	No	Yes (for <code>native-abi</code>)

§ 7. Our Recommendation

We recommend adopting both solutions in combination: mandating array-like layout for `native-abi` (Solution 1) and providing query traits for all ABIs (Solution 2).

The historical evidence is compelling: essentially every major CPU architecture in widespread use over the last 25 years exposes SIMD facilities whose interaction with memory is consistent with array-like element layout, from phones to servers. Scoping the mandate to `native-abi<T>` makes the guarantee precise and defensible: we are mandating layout for the ABI that represents the hardware’s native representation.

This scoping eliminates the trailing padding concern entirely. A `native-abi` vector maps to a single hardware register, and the elements fill it completely. The object representation is identical to `array<T, size()>`, with no caveats or escape hatches.

The standard already assumes array-like layout for `native-abi` through multiple mechanisms: the meaning of "native," the recommended practice for conversions to intrinsics, and the existence of ABI tags for handling variations. Leaving layout unspecified creates an internal inconsistency when the indirect path through intrinsics is well-defined but direct `bit_cast` is not.

Providing the `is_simd_array_like_v` trait alongside the mandate extends the utility to generic code that must work across multiple ABIs. For `native-abi`, the trait is unconditionally `true` whilst for other ABIs, it provides the discoverability that enables adaptive algorithms without sacrificing correctness.

We recognize that the `fixed_size` problem remains open. Different implementations make legitimately different decomposition choices, and the question of pairwise layout compatibility between `fixed_size` specializations may prove more useful than per-type array-likeness. However, `native-abi` covers the vast majority of the motivating use cases — bit-manipulation idioms, type-punning between float and integer vectors of the same width — and provides a solid foundation on which to build.

We also recognize that this issue might be regarded as an evolution of `std::simd` rather than a defect fix. `bit_cast` is already legal, and an implementation could be reverse-engineered to determine its layout, but this adds overhead and complexity to the experience of programming with `std::simd`. The current state represents a usability regression compared to existing practice with vendor intrinsics, which have always had well-defined bit-casting semantics.

If the committee prefers not to mandate layout for `native-abi`, we offer Solution 2 alone as a fallback.

§ 8. Additional Discussion

§ 8.1. Implementation Experience

In high-performance software development, such as signal processing, it is extremely common to manipulate data at the bit level to achieve greater speed. At Intel we have large intrinsic-based software code bases where bit-casts are used frequently, and we have found that well-defined bit-casting semantics are essential for writing portable, high-performance code. While this software is written to perform well on Intel processors, customers increasingly demand portable code that runs well on multiple vendors' hardware and across a wide range of compilers. Using `std::simd` is an attractive way to achieve this portability, but not if it becomes impractical due to non-portable (implementation-defined) bit-casting behavior across targets and implementations.

§ 8.2. Scalar Type Consistency

If `int8_t` is 8 bits in scalar code, then `vec<int8_t, 8>` should store 8×8 -bit elements. Users reason about memory consumption, cache behavior, and bandwidth based on element size. If an implementation of `vec<int8_t, 16>` silently consumed 512 bits instead of 128 bits, it would defeat the purpose of using smaller types.

§ 8.3. Endianness

Mandating array-like layout for `basic_vec` does not introduce any endianness concerns beyond those that already exist for arrays. For a given element type `T`, each element's byte order in memory follows the platform's representation of `T`, exactly as it does for `array<T, N>`. Separately, the mapping from `basic_vec` element index to increasing memory offsets is defined by the proposed array-like layout (increasing index order), and this lane ordering is independent of the platform's endianness.

§ 8.4. Library Interoperability

BLAS, LAPACK, FFTW, Eigen, and game engines all assume array-like layout. Without a specified layout, `std::simd` cannot reliably interoperate with these libraries. Since library interop most commonly involves native-width vectors, the `native-abi` scope addresses the primary use case for such interoperability.

§ 8.5. ABI Stability

Since `std::simd` is new in C++26, there are no existing deployed standard-library implementations with established ABI contracts. Mandating array-like layout for `native-abi` therefore does not constitute an ABI break for any existing implementation. Furthermore, every implementation we are aware of already uses array-like layout for native-width vectors, so the mandate codifies existing practice rather than requiring changes.

§ 8.6. Complex Numbers

There has been historical debate about whether complex values should be stored in interleaved (real, imag, real, imag) or separated (all real, then all imag) format for SIMD processing. Both formats have advantages and disadvantages, leading to different choices for different problem domains or data sizes. However, the debate was ultimately resolved by hardware realities: modern hardware that supports

complex values (e.g., AVX-512 and ARM SVE) does so in interleaved form. This led even long-term proponents of separated storage, such as MATLAB, to switch to interleaved storage for their SIMD implementations. This historical precedent demonstrates that hardware realities tend to dominate in the long term, and that software must adapt to those realities for performance and interoperability.

§ 8.7. Masks

The `basic_mask` type presents the same general problem as `basic_vec`: the object representation is unspecified. However, masks differ from vectors because hardware genuinely diverges on mask representation. AVX2 uses full-element representation, where each boolean occupies the full element width (e.g., 32 bits for a mask corresponding to 32-bit elements). AVX-512 and ARM SVE use compact bit-mask representation, where each boolean is a single bit stored in a dedicated predicate register.

This divergence is not merely an implementation choice but it reflects fundamentally different hardware designs. Mandating a single representation for masks would disadvantage targets whose hardware uses the other format, unlike the vector case where all mainstream hardware agrees on array-like layout.

For this reason, the proposed `is_mask_array_like_v` trait allows users to query which representation a given mask specialization uses. This does not by itself make `std::bit_cast` to/from masks portable — that would additionally require specifying the exact mapping for each representation — but it provides the foundation for future work in this area.

§ 9. Proposed Wording

Wording is provided for the recommended combined approach and for Solution 2 alone as a fallback.

§ 9.1. Wording for Recommended Approach (Combined)

Modify `[simd.overview]` as follows:

The value representation of `basic_vec<T, native_abi<T>>` consists of `basic_vec<T, native_abi<T>>::size()` contiguously allocated values of type `T`, in increasing index order. Let `N` be `basic_vec<T, native_abi<T>>::size()`. The object representation of `basic_vec<T, native_abi<T>>` shall be identical to the object representation of `array<T, N>`. There shall be no padding between elements and no trailing padding.

[*Note*: When `std::bit_cast` between `basic_vec<T, native_abi<T>>` and `basic_vec<U, native_abi<U>>` is well-formed (i.e., both have the same size), it behaves equivalently to `std::bit_cast` between corresponding arrays. — *end note*]

For other ABIs, the object representation of `basic_vec<T, Abi>` is implementation-defined.

Modify [simd.mask.overview] to specify mask representation:

The value representation of `basic_mask<Bytes, Abi>` consists of `basic_mask<Bytes, Abi>::size()` boolean values stored either as full elements (each occupying `Bytes` bytes) or as compact bits (one bit per boolean), as determined by the implementation. The representation is consistent for a given `Abi`. There shall be no padding between elements in full-element representation.

[*Note*: Different hardware uses different mask representations. AVX2 uses full-element representation where each boolean occupies the full element size. AVX-512 and ARM SVE use compact bit-mask representation where each boolean is a single bit. — *end note*]

Add to [simd.traits]:

```
template<typename T, typename Abi>
inline constexpr bool is_simd_array_like_v = /* see below */;

template<size_t Bytes, typename Abi>
inline constexpr bool is_mask_array_like_v = /* see below */;
```

Returns: For `is_simd_array_like_v<T, Abi>`: true if the object representation of `basic_vec<T, Abi>` is identical to the object representation of `array<T, N>` (where `N` is `basic_vec<T, Abi>::size()`), with no padding between elements and no trailing padding, and elements are stored in increasing index order. Otherwise, false.

Remarks: `is_simd_array_like_v<T, native_abi<T>>` is true for all `T` for which `basic_vec<T, native_abi<T>>` is a valid specialization.

Returns: For `is_mask_array_like_v<Bytes, Abi>`: true if `basic_mask<Bytes, Abi>` uses full-element representation where each boolean element occupies `Bytes` contiguous bytes (with no padding between elements, and in increasing index order). Otherwise, false indicates a compact bitmask representation.

[*Note:* When `is_simd_array_like_v<T, Abi>` is true, `bit_cast` between `basic_vec` specializations behaves equivalently to `bit_cast` between corresponding arrays. The mask trait distinguishes between full-element and compact representations. — end note]

Add to [simd.expos.abi]:

For ABIs other than `native_abi`, the object representation of `basic_vec<T, Abi>` is implementation-defined. Implementations shall document whether `is_simd_array_like_v<T, Abi>` is true or false for each supported combination. Implementations shall document whether `is_mask_array_like_v<Bytes, Abi>` is true or false for each supported combination.

§ 9.2. Wording for Solution 2 Alone (Fallback)

If the committee prefers not to mandate layout for `native_abi`, the following provides query traits without any layout mandate.

Add to [simd.traits]:

```
template<typename T, typename Abi>
inline constexpr bool is_simd_array_like_v = /* see below */;

template<size_t Bytes, typename Abi>
inline constexpr bool is_mask_array_like_v = /* see below */;
```

Returns: For `is_simd_array_like_v<T, Abi>`: true if the object representation of `basic_vec<T, Abi>` is identical to the object representation of `array<T, N>` (where `N` is `basic_vec<T, Abi>::size()`), with no padding between elements and no trailing padding, and elements are stored in increasing index order. Otherwise, false.

Returns: For `is_mask_array_like_v<Bytes, Abi>`: true if `basic_mask<Bytes, Abi>` uses full-element representation. Otherwise, false indicates compact bitmask representation.

[*Note:* When `is_simd_array_like_v<T, Abi>` is true, `bit_cast` between `basic_vec` specializations behaves equivalently to `bit_cast` between corresponding arrays. The mask trait distinguishes between full-element and compact representations. — *end note*]

Add to [simd.expos.abi]:

The object representation of `basic_vec<T, Abi>` and `basic_mask<Bytes, Abi>` is implementation-defined. Implementations shall document whether `is_simd_array_like_v<T, Abi>` and `is_mask_array_like_v<Bytes, Abi>` are true or false for each supported combination.

§ 10. Impact on Existing Code

For users, the recommended approach means existing code relying on array-like layout for `native-abi` vectors continues to work, and code that avoided `bit_cast` can now use it portably with `native-abi`. The `is_simd_array_like_v` trait additionally enables generic code to adapt to other ABIs. There are no breaking changes.

For implementations, the mandate applies only to `native-abi`, which is already what every implementation we are aware of provides. Implementations using array-like layout for native-width vectors (Intel, GCC, Clang, on major platforms) require no changes. Implementations of `fixed_size` and custom ABIs are entirely unaffected, needing only to provide the query traits.

The fallback (Solution 2 alone) requires all implementations to document layout properties and provide query traits, with minimal code changes.

§ 11. Future Work

This paper deliberately limits its scope to `native-abi` layout and basic mask representation queries. Several related topics merit further investigation in future papers:

- **`fixed_size` layout specification.** The `fixed_size` ABI presents challenges due to differing register decomposition strategies across implementations. A future paper should explore whether a useful layout guarantee can be provided, potentially through constraints on how implementations decompose multi-register vectors.
- **Pairwise layout compatibility for `fixed_size`.** As discussed in §fixed-size, the most useful guarantee for `fixed_size` may be pairwise compatibility ("are these two types layout-compatible?") rather than per-type array-likeness. A future paper could propose a trait such as `is_simd_layout_compatible_v<T1, Abi1, T2, Abi2>` to express this relationship.
- **Mask representation guarantees for `native-abi`.** This paper provides `is_mask_array_like_v` as a query trait but does not mandate a specific mask representation, because hardware genuinely diverges. A future paper could specify the exact bit-level mapping for each representation, enabling portable `bit_cast` to and from masks.
- **Explicit reinterpretation functions.** Rather than relying solely on `std::bit_cast`, a future paper could propose dedicated functions such as `simd_reinterpret<To>(from)` that express type-punning intent directly, potentially with relaxed constraints (e.g., handling size mismatches by truncation or zero-extension).